Série: Descomplicando Kubernetes

Secrets e ConfigMaps



LINUXTIPS

Jeferson Fernando

Secrets e ConfigMaps

Este e-book é uma contribuição à comunidade open source de Kubernetes, produzido e distribuído gratuitamente em português pela LINUXtips. Esta edição beta aborda dois componentes essenciais do Kubernetes: Secrets e ConfigMaps. A versão final 1.0 será lançada em breve - para se manter atualizado, acompanhe o site da LINUXtips ou inscreva-se na lista de e-mails do Jeferson Fernando.

Este trabalho está licenciado sob GPL-3 e pode ser usado e distribuído respeitando as condições dessa licença. O e-book é um projeto colaborativo e convida você a contribuir com correções ou adições de novos conteúdos no repositório do GitHub: https://github.com/badtuxx/ebooks. Junte-se a nós nesse esforço para disseminar o conhecimento!

Resumo do Conteúdo

Este e-book fornece um guia abrangente para gerenciar Secrets e ConfigMaps no Kubernetes. Os Secrets são um recurso que permite gerenciar informações sensíveis como senhas, tokens OAuth, chaves ssh, entre outros. Aprendemos a criar, obter, descrever e deletar Secrets, e a proteger essas informações críticas usando codificação base64.

O e-book vai além, apresentando o uso de Secrets para armazenar certificados TLS e chaves privadas, configurando o Nginx para usar HTTPS. Aprenderemos como definir Secrets através de um arquivo de manifesto e como usá-los em um Pod do Nginx.

Também exploramos ConfigMaps, que permitem separar parâmetros de configuração das imagens do container, permitindo a mesma imagem ser executada em diferentes ambientes - como desenvolvimento, teste e produção - com configurações distintas. Vamos abordar como atualizar um ConfigMap, definir variáveis de ambiente para os containers no Pod, tornar ConfigMaps imutáveis, e criar um arquivo de configuração do Nginx usando um ConfigMap.

Por fim, este e-book demonstra como usar ConfigMaps e Secrets juntos para configurar um Pod do Nginx para usar HTTPS. Este material detalhado busca equipar os leitores com o conhecimento necessário para gerenciar eficientemente

suas configurações e dados sensíveis, proporcionando flexibilidade e controle precisos sobre suas aplicações.

Esperamos que você encontre valor neste material e o incentive a colaborar, melhorando ainda mais este e-book para a comunidade Kubernetes.

Use, Colabore e Compartilhe!

Este trabalho foi desenvolvido para disseminar conhecimento e incentivar a colaboração. Esperamos que você encontre valor nele e se sinta motivado a compartilhar o que aprendeu, ensinando outros e fortalecendo a comunidade.

Ao compartilhar nas redes sociais, marque Jeferson Fernando (@*badtux*_) e utilize a hashtag #KubernetesExpert. Estamos ansiosos para ver como este material inspirará a comunidade a aprender e a crescer juntos.

Além do E-book: Descomplicando o Kubernetes

Se você busca uma compreensão mais profunda e uma experiência prática com o Kubernetes, convidamos você a conhecer o curso "Descomplicando o Kubernetes". Ministrado por Jeferson Fernando, este é o curso mais renomado sobre Kubernetes, completamente atualizado e reestruturado para proporcionar uma aprendizagem ainda mais eficaz.

Com desafios práticos e reais, todo o conteúdo é ministrado em português. E como incentivo ao seu aprendizado aprofundado, utilize o código de desconto #ContainerExpert e obtenha 20% de desconto!

https://www.linuxtips.io/course/descomplicando-o-kubernetes

O que iremos ver hoje?

Hoje é dia de falar sobre duas coisas super importantes no mundo do Kubernetes, hoje nós vamos falar sobre `Secrets` e `ConfigMaps`.

Sim, essas duas peças são fundamentais para que você possa ter a sua aplicação rodando no Kubernetes da melhor forma possível, pois elas são responsáveis por armazenar as informações sensíveis da sua aplicação, como por exemplo, senhas, tokens, chaves de acesso, configurações, etc.

Depois do dia de hoje você vai entender como funciona o armazenamento de informações sensíveis no Kubernetes e como você pode fazer para que a sua aplicação possa consumir essas informações da melhor forma possível.

Bora lá?

O que são Secrets?

Os Secrets fornecem uma maneira segura e flexível de gerenciar informações sensíveis, como senhas, tokens OAuth, chaves SSH e outros dados que você não quer expor nas configurações de suas aplicações.

Um Secret é um objeto que contém um pequeno volume de informações sensíveis, como uma senha, um token ou uma chave. Essas informações podem ser consumidas por Pods ou usadas pelo sistema para realizar ações em nome de um Pod.

Como os Secrets funcionam

Os Secrets são armazenados no Etcd, o armazenamento de dados distribuído do Kubernetes. Por padrão, eles são armazenados sem criptografia, embora o Etcd suporte criptografia para proteger os dados armazenados nele. Além disso, o acesso aos Secrets é restrito por meio de Role-Based Access Control (RBAC), o que permite controlar quais usuários e Pods podem acessar quais Secrets.

Os Secrets podem ser montados em Pods como arquivos em volumes ou podem ser usados para preencher variáveis de ambiente para um container dentro de um Pod. Quando um Secret é atualizado, o Kubernetes não atualiza automaticamente os volumes montados ou as variáveis de ambiente que se referem a ele.

Tipos de Secrets

Existem vários tipos de Secrets que você pode usar, dependendo de suas necessidades específicas. Abaixo estão alguns dos tipos mais comuns de Secrets:

- Opaque Secrets - são os Secrets mais simples e mais comuns. Eles armazenam dados arbitrários, como chaves de API, senhas e tokens. Os Opaque Secrets são codificados em base64 quando são armazenados no Kubernetes, mas não são criptografados. Eles podem ser usados para armazenar dados confidenciais, mas não são seguros o suficiente para armazenar informações altamente sensíveis, como senhas de banco de dados.
- kubernetes.io/service-account-token - são usados para armazenar tokens de acesso de conta de serviço. Esses tokens são usados para autenticar Pods

com o Kubernetes API. Eles são montados automaticamente em Pods que usam contas de serviço.

- kubernetes.io/dockercfg e kubernetes.io/dockerconfigjson - são usados para armazenar credenciais de registro do Docker. Eles são usados para autenticar Pods com um registro do Docker e são montados em Pods que usam imagens de container privadas.
- kubernetes.io/tls, kubernetes.io/ssh-auth e kubernetes.io/basic-auth - são usados para armazenar certificados TLS, chaves SSH e credenciais de autenticação básica, respectivamente. Eles são usados para autenticar Pods com outros serviços.
- bootstrap.kubernetes.io/token - são usados para armazenar tokens de inicialização de cluster. Eles são usados para nos autenticar com o control plane do Kubernetes.

Tem mais alguns tipos de Secrets, mas esses que te passei são os mais comuns. Você pode encontrar uma lista completa de tipos de Secrets na documentação do Kubernetes.

Cada tipo de Secret tem um formato diferente. Por exemplo, os Secrets Opaque são armazenados como um mapa de strings, enquanto os Secrets TLS são armazenados como um mapa de strings com chaves adicionais para armazenar certificados e chaves, por isso é importante saber qual tipo de Secret você está usando para que você possa armazenar os dados corretamente.

Antes de criar um Secret, o Base64

Antes de começarmos a criar os nossos Secrets, precisamos entender o que é o Base64, pois esse é um assunto que sempre gera muitas dúvidas e sempre está presente quando falamos de Secrets.

Primeira coisa, Base64 é criptografia? Não, Base64 não é criptografia, Base64 é um esquema de codificação binária para texto que visa garantir que os dados binários possam ser enviados por canais que são projetados para lidar apenas com texto. Esta codificação ajuda a garantir que os dados permaneçam intactos sem modificação durante o transporte.

Base64 está comumente usado em várias aplicações, incluindo e-mail via MIME, armazenamento de senhas complexas, e muito mais.

A codificação Base64 converte os dados binários em uma string de texto ASCII. Essa string contém apenas caracteres que são considerados seguros para uso em URLs, o que torna a codificação Base64 útil para codificar dados que estão sendo enviados pela Internet.

No entanto, a codificação Base64 não é uma forma de criptografia e não deve ser usada como tal. Qualquer um que tenha acesso à string codificada pode facilmente decodificá-la e recuperar os dados originais. Entender isso é importante para que você não armazene informações sensíveis em um formato codificado em Base64, pois isso não é seguro.

Agora que você já sabe o que é o Base64, vamos ver como podemos codificar e decodificar uma string usando o Base64.

Para codificar uma string em Base64, você pode usar o comando `base64` no Linux. Por exemplo, para codificar a string `giropops` em Base64, você pode executar o seguinte comando:

```
echo -n 'giropops' | base64
```

O comando acima irá retornar a string `Z2lyb3BvcHM=`.

Para decodificar uma string em Base64, você pode usar o comando `base64` novamente, mas desta vez com a opção `-d`. Por exemplo, para decodificar a string `Z2lyb3BvcHM=` em Base64, você pode executar o seguinte comando:

```
echo -n 'Z2lyb3BvcHM=' | base64 -d
```

O comando acima irá retornar a string `giropops`, simples como voar!

Estou usando o parâmetro `-n` no comando `echo` para que ele não adicione uma nova linha ao final da string, pois isso pode causar problemas ao codificar e decodificar a string.

Pronto, acho que você já está pronto para criar os seus Secrets, então é hora de começar a brincar!

Criando nosso primeiro Secret

Agora que você já sabe o que são os Secrets, já entendeu que Base64 não é criptografia e já sabe como codificar e decodificar uma string usando o Base64, vamos criar o nosso primeiro Secret.

Primeiro, vamos criar um Secret do tipo Opaque. Este é o tipo de Secret mais comum, usado para armazenar informações arbitrárias.

Para criar um Secret do tipo Opaque, você precisa criar um arquivo YAML que define o Secret. Por exemplo, você pode criar um arquivo chamado `giropops-secret.yaml` com o seguinte conteúdo:

```
apiVersion: v1
kind: Secret
metadata:
  name: giropops-secret
type: Opaque
data: # Inicio dos dados
    username: SmVmZXJzb25fbGluZG8=
    password: Z2lyb3BvcHM=
```

O arquivo acima define um Secret chamado `giropops-secret` com dois campos de dados chamados `username` e `password`. O campo `password` contém a string `giropops` codificada em Base64. A minha pergunta é: qual é o valor do campo `username`?

Caso você descubra, eu desafio você a fazer um Tweet ou um post em outra rede social com o valor do campo `username` e as hashtags #desafioDay8 e #DescomplicandoKubernetes. Se você fizer isso, eu vou te dar um prêmio, mas não vou falar qual é o prêmio, pois isso é um segredo! :D

Outra informação importante que passamos para esse Secret foi o seu tipo, que é `Opaque`. Você pode ver que o tipo do Secret é definido no campo `type` do arquivo YAML.

Agora que você já criou o arquivo YAML, você pode criar o Secret usando o comando `kubectl create` ou `kubectl apply`. Por exemplo, para criar o Secret usando o comando `kubectl create`, vá com o seguinte comando:

```
kubectl create -f giropops-secret.yaml

secret/giropops-secret created
```

Pronto, o nosso primeiro Secret foi criado! Agora você pode ver o Secret usando o comando `kubectl get`:

```
kubectl get secret giropops-secret

NAME              TYPE      DATA   AGE
giropops-secret   Opaque    2      10s
```

A saída traz o nome do Secret, o seu tipo, a quantidade de dados que ele armazena e a quanto tempo ele foi criado.

Caso você queira ver os dados armazenados no Secret, você pode usar o comando `kubectl get` com a opção `-o yaml`:

```
kubectl get secret giropops-secret -o yaml

apiVersion: v1
data:
  password: Z2lyb3BvcHM=
  username: SmVmZXJzb25fbGluZG8=
kind: Secret
metadata:
  creationTimestamp: "2023-05-21T10:38:39Z"
  name: giropops-secret
  namespace: default
  resourceVersion: "466"
  uid: ac816e95-8896-4ad4-9e64-4ee8258a8cda
type: Opaque
```

Simples assim! Portanto, mais uma vez, os dados armazenados no Secret não são criptografados e podem ser facilmente decodificados por qualquer pessoa que tenha acesso ao Secret, portanto, é fundamental controlar o acesso aos Secrets e não armazenar informações sensíveis neles.

Você também pode ver os detalhes do Secret usando o comando `kubectl describe`:

```
kubectl describe secret giropops-secret

Name:         giropops-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  8 bytes
username:  15 bytes
```

A saída do comando `kubectl describe` é muito parecido com o conteúdo do arquivo YAML que você criou para definir o Secret, mas com algumas informações

adicionais, como o namespace do Secret, os labels e as annotations, coisas que você também pode definir no arquivo YAML.

Caso você queira criar esse mesmo Secret usando o comando `kubectl create secret`, você pode executar o seguinte comando:

```
kubectl create secret generic giropops-secret --from-literal=username=<SEGREDO>
--from-literal=password=giropops
```

Fácil, aqui estamos usando o parâmetro `--from-literal` para definir os dados do Secret. Outras opções são `--from-file` e `--from-env-file`, que você pode usar para definir os dados do Secret a partir de um arquivo ou de variáveis de ambiente.

Se você comparar as strings dos campos `username` e `password` do Secret criado usando o comando `kubectl create secret` com as strings dos campos `username` e `password` do Secret criado usando o arquivo YAML, você perceberá que são iguais. Isso acontece porque o comando `kubectl create secret` codifica os dados em Base64 automaticamente.

Usando o nosso primeiro Secret

Agora que já temos o nosso primeiro Secret criado, é hora de saber como usá-lo em um Pod.

Nesse nosso primeiro exemplo, somente irei mostrar como usar o Secret em um Pod, mas ainda sem nenhuma "função" especial, apenas para mostrar como usar o Secret.

Para usar o Secret em um Pod, você precisa definir o campo `spec.containers[].env[].valueFrom.secretKeyRef` no arquivo YAML do Pod. Eu estou trazendo o campo nesse formato, para que você possa começar a se familiarizar com esse formato, pois você irá usá-lo bastante para buscar alguma informação mais específica na linha de comando, usando o comando `kubectl get`, por exemplo.

Voltando ao assunto principal, precisamos criar o nosso Pod, então vamos lá! Crie um arquivo chamado `giropops-pod.yaml` com o seguinte conteúdo:

```
apiVersion: v1
kind: Pod
metadata:
  name: giropops-pod
spec:
  containers:
```

```
  - name: giropops-container
    image: nginx
    env: # Início da definição das variáveis de ambiente
    - name: USERNAME # Nome da variável de ambiente que será usada no Pod
      valueFrom: # Início da definição de onde o valor da variável de ambiente
será buscado
        secretKeyRef: # Início da definição de que o valor da variável de ambiente
será buscado em um Secret, através de uma chave
          name: giropops-secret # Nome do Secret que contém o valor da variável de
ambiente que será usada no Pod
          key: username # Nome da chave do campo do Secret que contém o valor da
variável de ambiente que será usada no Pod
    - name: PASSWORD # Nome da variável de ambiente que será usada no Pod
      valueFrom: # Início da definição de onde o valor da variável de ambiente
será buscado
        secretKeyRef: # Início da definição de que o valor da variável de ambiente
será buscado em um Secret, através de uma chave
          name: giropops-secret # Nome do Secret que contém o valor da variável de
ambiente que será usada no Pod
          key: password # Nome da chave do campo do Secret que contém o valor da
variável de ambiente que será usada no Pod
```

Eu adicionei comentários nas linhas que são novas para você, para que você possa entender o que cada linha faz.

Mas vou trazer aqui uma explicação mais detalhada sobre o campo `spec.containers[].env[].valueFrom.secretKeyRef`:

- `spec.containers[].env[].valueFrom.secretKeyRef.name`: o nome do Secret que contém o valor da variável de ambiente que será usada no Pod;
- `spec.containers[].env[].valueFrom.secretKeyRef.key`: a chave do campo do Secret que contém o valor da variável de ambiente que será usada no Pod;

Com isso teremos um Pod, que terá um container chamado `giropops-container`, que terá duas variáveis de ambiente, `USERNAME` e `PASSWORD`, que terão os valores que estão definidos no Secret `giropops-secret`.

Agora vamos criar o Pod usando o comando `kubectl apply`:

```
kubectl apply -f giropops-pod.yaml

pod/giropops-pod created
```

Agora vamos verificar se o Pod foi criado e se os Secrets foram injetados no Pod:

```
kubectl get pods

NAME           READY   STATUS    RESTARTS   AGE
giropops-pod   1/1     Running   0          2m
```

Para verificar se os Secrets foram injetados no Pod, você pode usar o comando `kubectl exec` para executar o comando `env` dentro do container do Pod:

```
kubectl exec giropops-pod -- env

PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=giropops-pod
NGINX_VERSION=1.23.4
NJS_VERSION=0.7.11
PKG_RELEASE=1~bullseye
PASSWORD=giropops
USERNAME=CENSURADO
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
HOME=/root
```

Olha lá os nossos Secrets como variáveis de ambiente dentro do container do Pod!

Pronto! Tarefa executada com sucesso! \o/

Agora eu acho que já podemos partir para os próximos tipos de Secrets!

Criando um Secret para armazenar credenciais Docker

O Docker Hub é um serviço de registro de imagens Docker, que permite que você armazene e compartilhe imagens Docker publicamente ou privadamente. Em 2022, o Docker Hub começou a limitar o número de downloads de imagens Docker públicas para 100 downloads por 6 horas para usuários não autenticados, e para usuários autenticados, o limite é de 200 downloads por 6 horas.

Mas o ponto aqui é que você pode usar o Docker Hub para armazenar imagens Docker privadas, e para isso você precisa de uma conta no Docker Hub, e para

acessar a sua conta no Docker Hub, você precisa de um nome de usuário e uma senha. Entendeu onde eu quero chegar? :D

Para que o Kubernetes possa acessar o Docker Hub, você precisa criar um Secret que armazene o nome de usuário e a senha da sua conta no Docker Hub, e depois você precisa configurar o Kubernetes para usar esse Secret.

Quando você executa `docker login` e tem a sua autenticação bem sucedida, o Docker cria um arquivo chamado `config.json` no diretório `~/.docker/` do seu usuário, e esse arquivo contém o nome de usuário e a senha da sua conta no Docker Hub, e é esse arquivo que você precisa usar para criar o seu Secret.

Primeiro passo é pegar o conteúdo do seu arquivo `config.json` e codificar em base64, e para isso você pode usar o comando `base64`:

```
base64 ~/.docker/config.json

QXF1aSB0ZW0gcXVlIGVzdGFyIG8gY29udGXDumRvIGRvIHNldSBjb25maWcuanNvbiwgY29pc2EgbGluZG
EgZG8gSmVmaW0=
```

Então vamos lá! Crie um arquivo chamado `dockerhub-secret.yaml` com o seguinte conteúdo:

```
apiVersion: v1
kind: Secret
metadata:
  name: docker-hub-secret # nome do Secret
type: kubernetes.io/dockerconfigjson # tipo do Secret, neste caso é um Secret que
armazena credenciais Docker
data:
  .dockerconfigjson: |  # substitua este valor pelo conteúdo do seu arquivo
config.json codificado em base64

QXF1aSB0ZW0gcXVlIGVzdGFyIG8gY29udGXDumRvIGRvIHNldSBjb25maWcuanNvbiwgY29pc2EgbGluZG
EgZG8gSmVmaW0=
```

O que temos de novo aqui é no campo `type`, que define o tipo do Secret, e neste caso é um Secret que armazena credenciais Docker, e no campo `data` temos o campo `dockerconfigjson`, que é o nome do campo do Secret que armazena o conteúdo do arquivo `config.json` codificado em base64.

Agora vamos criar o Secret usando o comando `kubectl apply`:

```
kubectl apply -f dockerhub-secret.yaml
```

```
secret/docker-hub-secret created
```

Para listar o Secret que acabamos de criar, você pode usar o comando `kubectl get`:

```
kubectl get secrets

NAME                TYPE                                  DATA   AGE
docker-hub-secret   kubernetes.io/dockerconfigjson   1      1s
```

Secret criada, agora já podemos testar o acesso ao Docker Hub!

Agora o Kubernetes já tem acesso ao Docker Hub, e você pode usar o Kubernetes para fazer o pull de imagens Docker privadas do Docker Hub.

Um coisa importante, sempre quando você precisar criar um Pod que precise utilizar uma imagem Docker privada do Docker Hub, você precisa configurar o Pod para usar o Secret que armazena as credenciais do Docker Hub, e para isso você precisa usar o campo `spec.imagePullSecrets` no arquivo YAML do Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: meu-pod
spec:
  containers:
  - name: meu-container
    image: minha-imagem-privada
  imagePullSecrets: # campo que define o Secret que armazena as credenciais do
Docker Hub
  - name: docker-hub-secret # nome do Secret
```

Perceba a utilização do campo `spec.imagePullSecrets` no arquivo YAML do Pod, e o campo `name` que define o nome do Secret que armazena as credenciais do Docker Hub. É somente isso que você precisa fazer para que o Kubernetes possa acessar o Docker Hub.

Criando um Secret TLS

O Secret `kubernetes.io/tls`, é usado para armazenar certificados TLS e chaves privadas. Eles são usados para fornecer segurança na comunicação entre os serviços no Kubernetes. Por exemplo, você pode usar um Secret TLS para configurar o HTTPS no seu serviço web.

Para criar um Secret TLS, você precisa ter um certificado TLS e uma chave privada, e você precisa codificar o certificado e a chave privada em base64, para então criar o Secret.

Vamos criar um Secret TLS para o nosso serviço web, mas para isso, você precisa ter um certificado TLS e uma chave privada antes de mais nada.

Para criar um certificado TLS e uma chave privada, você pode usar o comando `openssl`:

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout chave-privada.key -out certificado.crt
```

No comando acima, estamos criando um certificado TLS e uma chave privada, e o certificado e a chave privada serão armazenados nos arquivos `certificado.crt` e `chave-privada.key`, respectivamente. Você pode substituir os nomes dos arquivos por qualquer nome que você quiser. Estamos usando o comando `openssl` para criar um certificado TLS auto-assinado, e para isso você precisa responder algumas perguntas, como o país, estado, cidade, etc. Você pode responder qualquer coisa, não tem problema. Esse certificado TLS auto-assinado é apenas para fins de teste, e não deve ser usado em produção. Estamos passando o parâmetro `-nodes` para que a chave privada não seja criptografada com uma senha, e o parâmetro `-days` para definir a validade do certificado TLS, que neste caso é de 365 dias. Já o parâmetro `-newkey` é usado para definir o algoritmo de criptografia da chave privada, que neste caso é o `rsa:2048`, que é um algoritmo de criptografia assimétrica que usa chaves de 2048 bits.

Eu não quero entrar em detalhes sobre como o que é um certificado TLS e uma chave privada, mas, basicamente, um certificado TLS (Transport Layer Security) é usado para autenticar e estabelecer uma conexão segura entre duas partes, como um cliente e um servidor. Ele contém informações sobre a entidade para a qual foi emitido e a entidade que o emitiu, bem como a chave pública da entidade para a qual foi emitido.

A chave privada, por outro lado, é usada para descriptografar a informação que foi criptografada com a chave pública. Ela deve ser mantida em segredo e nunca compartilhada, pois qualquer pessoa com acesso à chave privada pode decifrar a

comunicação segura. Juntos, o certificado TLS e a chave privada formam um par de chaves que permite a autenticação e a comunicação segura entre as partes.

Entendido? Espero que sim, porque eu não vou entrar em mais detalhes sobre isso. hahaha

Agora vamos voltar o foco na criação do Secret TLS.

Com o certificado TLS e a chave privada criados, vamos criar o nosso Secret, é somente para mudar um pouco, vamos criar o Secret usando o comando `kubectl apply`:

```
kubectl create secret tls meu-servico-web-tls-secret --cert=certificado.crt
--key=chave-privada.key

secret/meu-servico-web-tls-secret created
```

Vamos ver se o Secret foi criado:

```
kubectl get secrets
NAME                          TYPE                                  DATA   AGE
meu-servico-web-tls-secret    kubernetes.io/tls                     2      4s
```

Sim, o Secret está lá e é do tipo `kubernetes.io/tls`.

Caso você queira ver o conteúdo do Secret, você pode usar o comando `kubectl get secret` com o parâmetro `-o yaml`:

```
kubectl get secret meu-servico-web-tls-secret -o yaml
```

Agora você pode usar esse Secret para ter o Nginx rodando com HTTPS, e para isso você precisa usar o campo `spec.tls` no arquivo YAML do Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx
```

```
    ports:
      - containerPort: 80
      - containerPort: 443
    volumeMounts:
      - name: nginx-config-volume
        mountPath: /etc/nginx/nginx.conf
        subPath: nginx.conf
      - name: nginx-tls
        mountPath: /etc/nginx/tls
  volumes:
  - name: nginx-config-volume
    configMap:
      name: nginx-config
  - name: nginx-tls
    secret:
      secretName: meu-servico-web-tls-secret
      items:
        - key: certificado.crt
          path: certificado.crt
        - key: chave-privada.key
          path: chave-privada.key
```

Aqui temos bastante informação nova, então vamos por partes.

Primeira coisa que temos que falar é sobre o `spec.containers`, principalmente sobre os volumes, que é o campo `spec.containers.volumeMounts`.

O campo `spec.containers.volumeMounts` é usado para montar um volume em um diretório dentro do container. No nosso caso, estamos montando dois volumes, um para o arquivo de configuração do Nginx, e outro para o certificado TLS e a chave privada.

E usamos o campo `spec.volumes` para definir os volumes que serão usados pelo Pod, e estamos definindo dois volumes, o `nginx-config-volume` e o `nginx-tls`.

O volume `nginx-config-volume` é um volume do tipo `configMap`, e ele é usado para montar o arquivo de configuração do Nginx, que está armazenado no ConfigMap `nginx-config`. O próximo tópico é sobre ConfigMaps, então não se preocupe com isso agora.

Já o volume `nginx-tls` é um volume do tipo `secret`, e ele é usado para montar o Secret `meu-servico-web-tls-secret`, que contém o certificado TLS e a chave privada que serão usados para configurar o HTTPS no Nginx.

E como estamos configurando um Nginx para usar o nosso Secret, precisamos falar onde queremos que os arquivos do Secret sejam montados, e para isso usamos o

campo `spec.containers.volumeMounts.path` para definir o diretório onde queremos que os arquivos do Secret sejam montados, que neste caso é o diretório `/etc/nginx/tls`.

Falei que o volume `nginx-config-volume`, é um volume do tipo `configMap`, isso é uma ótima deixa para eu iniciar o próximo tópico, que é sobre ConfigMaps! :D

Sendo assim, bora continuar o nosso exemplo de como usar o Nginx com HTTPS, mas no próximo tópico sobre ConfigMaps. \o/

ConfigMaps

ConfigMaps são usados para armazenar dados de configuração, como variáveis de ambiente, arquivos de configuração, etc. Eles são muito úteis para armazenar dados de configuração que podem ser usados por vários Pods.

Os ConfigMaps são uma maneira eficiente de desacoplar os parâmetros de configuração das imagens de container. Isso permite que você tenha a mesma imagem de container em diferentes ambientes, como desenvolvimento, teste e produção, com diferentes configurações.

Aqui estão alguns pontos importantes sobre o uso de ConfigMaps no Kubernetes:

- Atualizações: Os ConfigMaps não são atualizados automaticamente nos pods que os utilizam. Se você atualizar um ConfigMap, os pods existentes não receberão a nova configuração. Para que um pod receba a nova configuração, você precisa recriar o pod.
- Múltiplos ConfigMaps: É possível usar múltiplos ConfigMaps para um único pod. Isso é útil quando você tem diferentes aspectos da configuração que quer manter separados.
- Variáveis de ambiente: Além de montar o ConfigMap em um volume, também é possível usar o ConfigMap para definir variáveis de ambiente para os containers no pod.
- Imutabilidade: A partir da versão 1.19 do Kubernetes, é possível tornar ConfigMaps (e Secrets) imutáveis, o que pode melhorar o desempenho de sua cluster se você tiver muitos ConfigMaps ou Secrets.

Como no exemplo do capítulo anterior, onde criamos um Pod com o Nginx, e usamos um ConfigMap para armazenar o arquivo de configuração do Nginx, o `ConfigMap` é usado para armazenar o arquivo de configuração do Nginx, ao invés de armazenar o arquivo de configuração dentro do Pod, tendo assim um Pod mais limpo e mais fácil de manter. E claro, sempre é bom usar as coisas para o que elas foram feitas, e o ConfigMap foi feito para armazenar dados de configuração.

Bora continuar o nosso exemplo de como usar o Nginx com HTTPS, mas agora usando um ConfigMap para armazenar o arquivo de configuração do Nginx.

Vamos criar o arquivo de configuração do Nginx chamado `nginx.conf`, que vai ser usado pelo ConfigMap:

```
events { } # configuração de eventos

http { # configuração do protocolo HTTP, que é o protocolo que o Nginx vai usar
  server { # configuração do servidor
    listen 80; # porta que o Nginx vai escutar
```

```
    listen 443 ssl; # porta que o Nginx vai escutar para HTTPS e passando o
parâmetro ssl para habilitar o HTTPS

    ssl_certificate /etc/nginx/tls/certificado.crt; # caminho do certificado TLS
    ssl_certificate_key /etc/nginx/tls/chave-privada.key; # caminho da chave
privada

    location / { # configuração da rota /
      return 200 'Bem-vindo ao Nginx!\n'; # retorna o código 200 e a mensagem
Bem-vindo ao Nginx!
      add_header Content-Type text/plain; # adiciona o header Content-Type com o
valor text/plain
    }
  }
}
```

Eu deixei o conteúdo do arquivo acima com comentários, para facilitar o entendimento.

O que o arquivo acima está fazendo é:

- Configurando o Nginx para escutar as portas 80 e 443, sendo que a porta 443 vai ser usada para o HTTPS.
- Configurando o Nginx para usar o certificado TLS e a chave privada que estão no diretório `/etc/nginx/tls`.
- Configurando a rota `/` para retornar o código 200 e a mensagem `Bem-vindo ao Nginx!` com o header `Content-Type` com o valor `text/plain`.

Agora vamos criar o ConfigMap `nginx-config` com o arquivo de configuração do Nginx:

```
kubectl create configmap nginx-config --from-file=nginx.conf
```

Simples demais, não? :)

O que estamos fazendo é criar um ConfigMap chamado `nginx-config` com o conteúdo do arquivo `nginx.conf`. Podemos fazer a mesma coisa através de um manifesto, como no exemplo abaixo:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
data:
  nginx.conf: |
```

```
    events { }

    http {
      server {
        listen 80;
        listen 443 ssl;

        ssl_certificate /etc/nginx/tls/certificado.crt;
        ssl_certificate_key /etc/nginx/tls/chave-privada.key;

        location / {
          return 200 'Bem-vindo ao Nginx!\n';
          add_header Content-Type text/plain;
        }
      }
    }
```

O arquivo é bem parecido com os manifestos do `Secret`, mas com algumas diferenças:

- O campo `kind` é `ConfigMap` ao invés de `Secret`.
- O campo `data` é usado para definir o conteúdo do ConfigMap, e o campo `data` é um mapa de chave-valor, onde a chave é o nome do arquivo e o valor é o conteúdo do arquivo. Usamos o caractere `|` para definir o valor do campo `data` como um bloco de texto, e assim podemos definir o conteúdo do arquivo `nginx.conf` sem a necessidade de usar o caractere `\n` para quebrar as linhas do arquivo.

Agora é só aplicar o manifesto acima:

```
kubectl apply -f nginx-config.yaml
```

Para ver o conteúdo do ConfigMap que criamos, bastar executar o comando:

```
kubectl get configmap nginx-config -o yaml
```

Você também pode usar o comando `kubectl describe configmap nginx-config` para ver o conteúdo do ConfigMap, mas o comando `kubectl get configmap nginx-config -o yaml` é bem mais completo.

```
kubectl describe configmap nginx-config

kubectl get configmap --all-namespaces

kubectl get configmap nginx-config -o yaml
```

Agora que já temos o nosso `ConfigMap` criado, vamos aplicar o manifesto que criamos no capítulo anterior, vou colar aqui o manifesto para facilitar:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
    - containerPort: 443
    volumeMounts:
    - name: nginx-config-volume # nome do volume que vamos usar para montar o
arquivo de configuração do Nginx
      mountPath: /etc/nginx/nginx.conf # caminho onde o arquivo de configuração do
Nginx vai ser montado
      subPath: nginx.conf # nome do arquivo de configuração do Nginx
    - name: nginx-tls # nome do volume que vamos usar para montar o certificado
TLS e a chave privada
      mountPath: /etc/nginx/tls # caminho onde o certificado TLS e a chave privada
vão ser montados
  volumes: # lista de volumes que vamos usar no Pod
  - name: nginx-config-volume # nome do volume que vamos usar para montar o
arquivo de configuração do Nginx
    configMap: # tipo do volume que vamos usar
      name: nginx-config # nome do ConfigMap que vamos usar
  - name: nginx-tls # nome do volume que vamos usar para montar o certificado TLS
e a chave privada
    secret: # tipo do volume que vamos usar
      secretName: meu-servico-web-tls-secret # nome do Secret que vamos usar
      items: # lista de arquivos que vamos montar, pois dentro da secret temos
dois arquivos, o certificado TLS e a chave privada
        - key: tls.crt # nome do arquivo que vamos montar, nome que está no campo
`data` do Secret
```

```
          path: certificado.crt # nome do arquivo que vai ser montado, nome que vai ser usado no campo `ssl_certificate` do arquivo de configuração do Nginx
        - key: tls.key # nome do arquivo que vamos montar, nome que está no campo `data` do Secret
          path: chave-privada.key # nome do arquivo que vai ser montado, nome que vai ser usado no campo `ssl_certificate_key` do arquivo de configuração do Nginx
```

Agora é só aplicar o manifesto acima:

```
kubectl apply -f nginx.yaml
```

Listando os Pods:

```
kubectl get pods
```

Agora precisamos criar um Service para expor o Pod que criamos:

```
kubectl expose pod nginx
```

Listando os Services:

```
kubectl get services
```

Bora fazer o `port-forward` para testar se o nosso Nginx está funcionando:

```
kubectl port-forward service/nginx 4443:443
```

O comando acima vai fazer o `port-forward` da porta 443 do Service `nginx` para a porta 4443 do seu computador, o `port-forward` salvando a nossa vida novamente! :)

Vamos usar o `curl` para testar se o nosso Nginx está funcionando:

```
curl -k https://localhost:4443

Bem-vindo ao Nginx!
```

Funcionando lindamente! Lembre-se que esse é um exemplo bem simples, o objetivo aqui é mostrar como usar o ConfigMap e o Secret para montar arquivos dentro de um Pod. O certificado TLS e a chave privada que usamos aqui são auto-assinados, e não são recomendados para uso em produção e não são aceitos pelos navegadores, mas para testar está ótimo.

Acho que já deu para entender como funciona o ConfigMap, e lembre-se que é possível usar o ConfigMap para montar arquivos, mas também é possível usar o ConfigMap para definir variáveis de ambiente, e isso é muito útil quando você precisa passar uma configuração para um container através de uma variável de ambiente.

Caso você queira tornar um ConfigMap imutável, você pode usar o campo `immutable` no manifesto do ConfigMap, como no exemplo abaixo:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  immutable: true # torna o ConfigMap imutável
data:
  nginx.conf: |
    events { }

    http {
      server {
        listen 80;
        listen 443 ssl;

        ssl_certificate /etc/nginx/tls/certificado.crt;
        ssl_certificate_key /etc/nginx/tls/chave-privada.key;

        location / {
          return 200 'Bem-vindo ao Nginx!\n';
          add_header Content-Type text/plain;
        }
      }
    }
```

Com isso, não será possível alterar o ConfigMap, e se você tentar alterar o ConfigMap, o Kubernetes vai retornar um erro.

Caso você queira deixar o ConfigMap em uma namespace específica, você pode usar o campo `namespace` no manifesto do ConfigMap, como no exemplo abaixo:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-config
  namespace: minha-namespace # deixa o ConfigMap na namespace `minha-namespace`
data:
  nginx.conf: |
    events { }

    http {
      server {
        listen 80;
        listen 443 ssl;

        ssl_certificate /etc/nginx/tls/certificado.crt;
        ssl_certificate_key /etc/nginx/tls/chave-privada.key;

        location / {
          return 200 'Bem-vindo ao Nginx!\n';
          add_header Content-Type text/plain;
        }
      }
    }
```

Enfim, acho que já vimos bastante coisa sobre ConfigMap, acho que já podemos ir para o próximo assunto, certo? \o/

Final do dia de hoje

Hoje o dia foi dedicado a dois componentes importantes do Kubernetes: Secrets e ConfigMaps.

Secrets no Kubernetes são um recurso que nos permite gerenciar informações sensíveis, como senhas, tokens OAuth, chaves ssh, etc. Devido à sua natureza sensível, o Kubernetes oferece uma série de recursos para gerenciar Secrets de maneira segura. Usamos o recurso base64 para codificar nossas senhas e chaves secretas. Aprendemos como criar, obter e descrever um Secret, bem como como excluir um Secret.

Fomos além e usamos o Secret para armazenar um certificado TLS e uma chave privada, que usamos para configurar o Nginx para usar HTTPS. Usamos o Secret para montar o certificado TLS e a chave privada em um Pod do Nginx, e usamos um arquivo de manifesto para definir o Secret.

Depois disso, exploramos ConfigMaps. ConfigMaps são uma maneira eficiente de separar parâmetros de configuração de imagens de container, permitindo que você tenha a mesma imagem de container rodando em diferentes ambientes como desenvolvimento, teste e produção, mas com configurações diferentes.

Vimos:

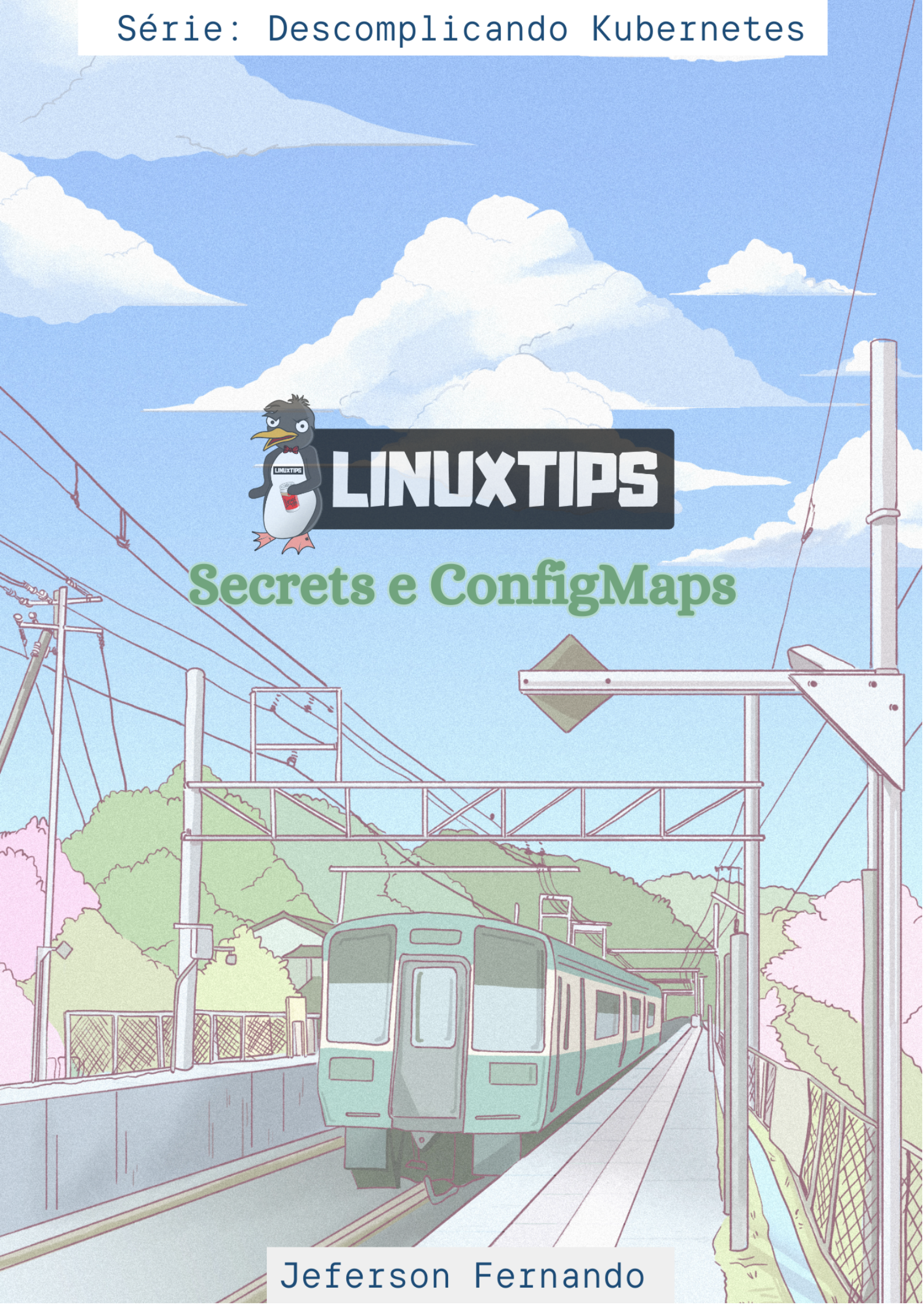
- Atualizar um ConfigMap.
- Usar o ConfigMaps.
- Usar o ConfigMap para definir variáveis de ambiente para os containers no Pod.
- Tornar ConfigMaps imutáveis.
- Criamos um arquivo de configuração do Nginx usando um ConfigMap, que usamos para configurar um Pod do Nginx. Também exploramos como montar o ConfigMap em um volume e como usar um arquivo de manifesto para definir o ConfigMap.

Finalmente, usamos o ConfigMap e o Secret juntos para configurar um Pod do Nginx para usar HTTPS, onde o ConfigMap é usado para armazenar o arquivo de configuração do Nginx e o Secret é usado para armazenar o certificado TLS e a chave privada.

Essa combinação de ConfigMaps e Secrets não só nos permite gerenciar nossas configurações e dados sensíveis de maneira eficiente e segura, mas também nos oferece um alto grau de flexibilidade e controle sobre as nossas aplicações.

E isso é tudo por hoje, chega! #VAIIII

Série: Descomplicando Kubernetes



Secrets e ConfigMaps

Jeferson Fernando